

```

0000 separate (IDL.SEM_PHASE)
|-----|
; -- GET_BASE_STRUCT
function IS_STATIC_RANGE (A: TREE) return Boolean is
begin
return A.TY = DN_RANGE and then GET_STATIC_VALUE (D (AS_EXP1, A)) /= TREE_VOID and
then GET_STATIC_VALUE (D (AS_EXP2, A)) /= TREE_VOID;
end IS_STATIC_RANGE;

function IS_STATIC_SUBTYPE (A : TREE) return Boolean is
begin
if A.TY in CLASS_PRIVATE_SPEC then
return GET_BASE_STRUCT (A).TY in CLASS_SCALAR and then IS_STATIC_SUBTYPE (D (SM_TYPE_SPEC, A));
elsif A.TY = DN_INCOMPLETE then
return GET_BASE_STRUCT (A).TY in CLASS_SCALAR and then IS_STATIC_SUBTYPE (D (XD_FULL_TYPE_SPEC, A));
end if;
if A.TY not in CLASS_SCALAR then
return False;
end if;
if D (SM_BASE_TYPE, A) = A then
-- THIS IS A SCALAR BASE TYPE; TEST-FOR GENERIC FORMAL TYPE
return D (SM_RANGE, A) /= TREE_VOID;
else
-- THIS A SUBTYPE, NOT A BASE TYPE
return IS_STATIC_RANGE (D (SM_RANGE, A)) and then IS_STATIC_SUBTYPE (D (SM_TYPE_SPEC, D (SM_RANGE, A)));
end if;
end IS_STATIC_SUBTYPE;

function IS_STATIC_DISCRETE_RANGE (A : TREE) return Boolean is
begin
case A.TY is
when DN_DISCRETE_SUBTYPE =>
return IS_STATIC_DISCRETE_RANGE (D (AS_SUBTYPE_INDICATION, A));
when DN_SUBTYPE_INDICATION =>
if D (AS_CONSTRAINT, A) = TREE_VOID then
return IS_STATIC_DISCRETE_RANGE (D (AS_NAME, A));
else
return IS_STATIC_DISCRETE_RANGE (D (AS_NAME, A)) and then IS_STATIC_RANGE (D (AS_CONSTRAINT, A));
end if;
when DN_RANGE =>
return IS_STATIC_RANGE (A);
when DN_USED_NAME_ID =>
return D (SM_DEFN, A) /= TREE_VOID and then IS_STATIC_DISCRETE_RANGE (D (SM_EXP_TYPE, D (SM_DEFN, A)));
when DN_SELECTED =>
return IS_STATIC_DISCRETE_RANGE (D (AS_DESIGNATOR, A));
when others =>
return False;
end case;
end IS_STATIC_DISCRETE_RANGE;

function IS_STATIC_INDEX_CONSTRAINT (ARRAY_TYPE, INDEX_CONSTRAINT : TREE) return Boolean is
INDEX_LIST := LIST (D (AS_DISCRETE_RANGE_S, INDEX_CONSTRAINT));
INDEX := TREE;
ARRAY_TYPE_SPEC := TREE;
ARRAY_TYPE := ARRAY_TYPE;
begin
while not IS_EMPTY (INDEX_LIST) loop
POP (INDEX_LIST, INDEX);
if not IS_STATIC_DISCRETE_RANGE (INDEX) then
return False;
end if;
if ARRAY_TYPE_SPEC.TY in CLASS_PRIVATE_SPEC then
ARRAY_TYPE_SPEC := D (SM_TYPE_SPEC, ARRAY_TYPE_SPEC);
elsif ARRAY_TYPE_SPEC.TY = DN_INCOMPLETE then
ARRAY_TYPE_SPEC := D (XD_FULL_TYPE_SPEC, ARRAY_TYPE_SPEC);
end if;
if ARRAY_TYPE_SPEC.TY = DN_CONSTRAINED_ARRAY then
-- $$$ SHOULD NOT HAPPEN (RM 3.6.1/3)
return False;
end if;
INDEX_LIST := LIST (D (SM_INDEX_S, ARRAY_TYPE_SPEC));
while not IS_EMPTY (INDEX_LIST) loop
POP (INDEX_LIST, INDEX);
if not IS_STATIC_SUBTYPE (D (SM_TYPE_SPEC, INDEX)) then
return False;
end if;
end loop;
return True;
end IS_STATIC_INDEX_CONSTRAINT;
|-----|
end CHK_STAT;

```